
Designing for Maintainability

Goal. Get a hands-on experience with real-world commit messages (and software maintenance in general) and think critically about them.

1 The Apache Arrow project

First, take a quick look at the Apache Arrow project: <https://github.com/apache/arrow>, which is a framework for developing data analytics applications.

2 Reading some git commit messages

Now, let's take a closer look at some of the commit messages in this project.

First, clone the project: `git clone https://github.com/apache/arrow`

Then, go to the repo: `cd arrow`

Now, check some recent commit messages: `git log -5 --before ``March 10 2023```. This gives 5 recent commits after the specified date. You can use your arrow keys to scroll up and down.

Please spend a few minutes reading these commit messages, pretending you're a maintainer who has to understand the codebase and these commits. Identify at least one strength and one weakness of each of the commit messages (e.g., in terms of its readability, whether it lists "why", etc.).

3 Ranking commit messages

Now, note down the strengths/weaknesses you have discovered. Feel free to discuss with your partners. Also consider ranking them in terms of which you would most rather see, if you were to maintain this codebase.

Commit 1 not good: Clearly states what part of the codebase was changed (e.g., "Fix crash in Python Flight client when closing connection"). The message is short and directly points to a specific component. Lacks an explanation of why the crash occurred or what conditions caused it. This makes it harder for a future maintainer to understand the underlying issue without inspecting the code.

Commit 2 ok: Includes both a concise summary and a linked issue number (e.g., "ARROW-15647: [C++] Optimize memory allocation in Parquet reader"). This allows easy traceability to the relevant discussion or bug report. The motivation behind the optimization ("why this change improves performance") is not explicitly mentioned.

Commit 3 good: The message describes both what was changed and partially why ("Add fallback for missing GPU acceleration to improve compatibility"). Including rationale helps others understand the design decision. No reference to testing or verification steps—maintainers might not know if the fallback was validated.

Commit 4 worst: Message includes details about documentation updates ("Update README and usage examples for new Arrow Dataset API"). Good practice for maintaining synchronization between code and docs. Slightly verbose; mixes unrelated small changes (both doc and minor refactor) that could have been separate commits for clarity.

Commit 5 best: Includes context and justification ("Revert change causing schema mismatch in Arrow Flight RPC"). The "why" is clear, showing the developer understood the problem introduced earlier. Could benefit from referencing the original faulty commit or issue number to improve traceability.

Lecture Notes:

Software maintenance is the dominant cost in the lifecycle of a system, so design choices should prioritize ease of maintenance from the start.

Investing extra effort during design reduces long-term maintenance costs — similar to building a “Forge” before constructing a “Wonder” in Civilization.

Maintenance can make up 70–90% of total software costs; increasing design effort by 50% is worthwhile if it halves maintenance effort.

Design should support easier comprehension, documentation, testing, change, and reuse — even if initial coding takes longer.

Shift mindset from “fast coding” to “readable, elegant, maintainable code.”

Code comprehension depends on essential complexity (inherent to the problem) and accidental complexity (from poor design).

Readability improves with shorter lines, consistent spacing, meaningful comments, moderate identifier variety, and blank lines where helpful.

Metrics can model readability, but using them blindly may create perverse incentives (e.g., adding excessive blank lines).

Documentation should capture why changes were made, not just what was done; tools can infer “what,” but only humans explain “why.”

High-quality comments and commit messages describe rationale, not just implementation details.

Object-oriented design principles help manage change: encapsulation hides volatile details, polymorphism enables extension, and contracts define clear expectations.

Delegation allows reuse by separating concerns—one object can rely on another for specific functionality without duplication.

Design patterns like Strategy and Template Method support change and reuse: Strategy via composition and interchangeable algorithms; Template Method via inheritance and invariant algorithm structure.

Design by contract enforces preconditions, postconditions, and invariants—subclasses must weaken preconditions and strengthen postconditions (Liskov Substitution Principle).

Testability should be designed in: modular architectures, unit testing interfaces, and fault injection mechanisms all make QA easier.

Adding controlled levels of indirection allows for testing and simulating errors safely.

Design for meaningful coverage—ensure code coverage correlates with requirement coverage by making implicit control flow explicit.

“Free tests” can come from reversible operations (e.g., $\text{deserialize}(\text{serialize}(x)) = x$).

In practice: review code and commit messages focusing on “why,” design systems to support testing and fault injection, and think like a maintainer, not just a coder.